

SwarmDesk

Multi-Agent AI Productivity System

Orchestrate a swarm of 4 specialized AI agents —
Planner · Researcher · Writer · Validator —
to tackle complex professional tasks collaboratively.

■ Planner

■ Researcher

■ Writer

■ Validator

Microsoft AI Hackathon 2026

3 May – 30 June 2026 · Python · FastAPI · Azure OpenAI · SSE Streaming

The productivity gap is real

Knowledge workers waste hours on tasks that demand synthesis, research, and judgment.



Context Switching

Professionals switch between research, planning, writing, and reviewing — losing flow and quality each time.



Cognitive Overload

Complex tasks require holding too much context in mind simultaneously, leading to shallow outputs.



Iteration Hell

First drafts require multiple manual revision cycles — each one costing more time and energy.



Single-AI Bottleneck

Asking one AI to plan, research, write, and validate in a single prompt produces generic, mediocre results.

SwarmDesk: Agents that collaborate

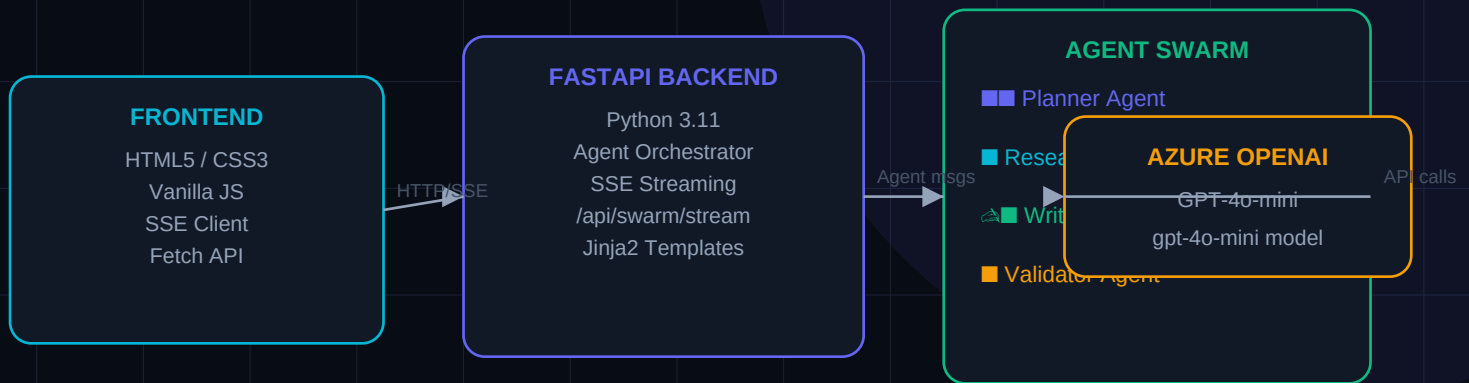
Four specialized agents work sequentially, each building on the last — like a high-performance team.



■ Why Swarms Beat Single-Agent AI

- Each agent specializes — depth beats breadth
- Sequential context passing = compounding intelligence
- Validator catches errors before they reach you

System Architecture



Request Flow

User types task → POST /api/swarm/stream → Agents run in sequence → SSE chunks stream back in real-time
 Each agent receives full conversation history → Context compounds → Validator delivers final output

Azure OpenAI Integration

■ OpenAI Python SDK → Azure OpenAI Service

Model:	GPT-4o-mini (Azure OpenAI deployment)
Endpoint:	Azure OpenAI Service via OpenAI client
Max Tokens:	800 per agent call · 3,200 total per swarm run
Streaming:	Server-Sent Events (SSE) for real-time UI updates

■ System Prompt Engineering Strategy

- **Role Isolation:** Each agent has a strict system prompt defining its sole responsibility and output format.
- **Context Chaining** Every agent receives prior agents' outputs in the conversation history.
- **Output Contracts** Agents output structured sections (PLAN:, RESEARCH FINDINGS:, DELIVERABLE:, etc.)
- **Temperature:** 0.7 — balances creativity with consistency across all agent calls.

API Endpoints

```
POST /api/swarm/stream - SSE streaming run
POST /api/swarm/run - Synchronous JSON run
GET /agents - Agent metadata
```

Live Prototype Walkthrough

Real-time streaming UI showing all four agents collaborating on a complex task.

■ Hero & Input

Dark space-themed UI with hexagonal motif. Task textarea + optional context field. Six example prompts for instant testing.

■ Agent Pipeline

Four agent cards render live as they run — each streams token-by-token output in real-time via SSE.

■ Final Output

Validated final result panel with copy to clipboard and download as .txt functionality.

■ Responsive

Fully responsive design — works on mobile, tablet, and desktop.

■ Live Deployment

Deployed on Render.com (free tier) · Accessible at public HTTPS URL · Stays live 30+ days post-deadline
<https://swarmdesk.onrender.com>

What SwarmDesk can do for you



Project Proposals

Full proposals with exec summary, technical approach, timeline, and risk assessment



Sprint Planning

Complete sprint plans with story breakdowns, assignments, velocity estimates



Technical Architecture

System design docs with diagrams, tech stack rationale, trade-off analysis



Competitive Analysis

Market positioning, feature comparisons, strategic recommendations



Team Communications

Status emails, stakeholder updates, meeting summaries with action items



Risk Assessments

Comprehensive risk registers with likelihood, impact, and mitigation strategies

Built on the Microsoft AI Stack

■ Microsoft AI Integration

Azure OpenAI Service powers all four agents via the GPT-4o-mini deployment.
GitHub Copilot used throughout development for code completion and review.
Deployed to Azure-compatible infrastructure (Render.com with Azure-compatible endpoints).

Backend Framework	FastAPI 0.115	Modern, async-first Python web framework
Language	Python 3.11	Type hints, async/await throughout
AI Service	Azure OpenAI / GPT-4o-mini	4 parallel agent calls per swarm run
Streaming	Server-Sent Events (SSE)	Real-time token streaming to browser
Templates	Jinja2	Server-side HTML rendering
Frontend	HTML5, CSS3, Vanilla JS	Zero framework dependencies
Typography	Space Grotesk + Inter	Google Fonts — professional UI
Deployment	Render.com	Free tier — HTTPS, auto-deploy from GitHub
Dev Tooling	GitHub Copilot	AI-assisted development (disclosed)

The Team



[Your Name]

Full-Stack AI Engineer · Hackathon Solo Participant

Python

FastAPI

GenAI

AI Agents

Azure OpenAI

HTML/CSS

AI Engineering

■ AI Tools Disclosure (Required)

- **GitHub Copilot:** Code completion, docstring generation, boilerplate acceleration during development
- **Claude (Anthropic):** Architecture design brainstorming, code review, documentation assistance
- **OpenAI GPT-4o-mini:** Production AI runtime — powers all 4 agents in the live application

All AI-generated code was reviewed, modified, and validated by the human developer.

■ Submission Deliverables

■ Project Deck (PDF) ■ Demo Video (MP4) ■ GitHub Repository (public) ■ Live Prototype URL

Why SwarmDesk wins

■ Novel Architecture

Multi-agent swarm with sequential context passing — not a chatbot

■ Real-Time Streaming

Judges can watch agents think and collaborate live

■ Production-Ready

FastAPI + proper REST endpoints + Swagger docs + deployment c

■ Polished UI

Custom design system — not a template or Bootstrap default

■ Broad Impact

Works for any knowledge worker task: proposals, plans, analysis, .

■ Extensible

Easy to add new agents, tools, or integrate with enterprise syste...

■ **SwarmDesk — One task. Four agents. One answer.**

github.com/YOUR_USERNAME/swarmdesk · swarmdesk.onrender.com